Multi-Sensor Fusion for Mobile Robot Localization Using a Hybrid EKF–Quantum Filtering Framework

Anish Kumar, Sachit Bansal, Somanshu Juneja
*U*nder the Guidance of Dr. Radhe Shyam

***Abstract*—Reliable localization of autonomous ground vehicles requires fusing complementary sensor modalities while handling measurement noise, nonlinear dynamics, and cumulative drift. This paper presents a hybrid Extended Kalman Filter–Quantum Filtering (EKF–QF) framework for multi-sensor data fusion on the Husarion Lynx unmanned ground vehicle (UGV). The system fuses five sensor modalities—wheel odometry, IMU, GPS, RTAB-Map RGB-D visual odometry [4], and RTAB-Map ICP LiDAR odometry—using the `robot_localization` ROS 2 package [11]. A Recurrent Quantum Neural Network (RQNN) [12] pre-filter with PSO-tuned hyperparameters selectively denoises IMU and visual odometry signals before EKF ingestion. Experiments in Gazebo simulation [17] demonstrate noise reduction exceeding 97% on IMU gyroscope X/Y channels, 72.3% on accelerometer X, and smoother fused trajectories relative to a standalone EKF baseline processing raw sensor data.**

***Index Terms*—Extended Kalman Filter, multi-sensor fusion, quantum filtering, RQNN, mobile robot localization, ROS 2, RTAB-Map, PSO**

I. Introduction

Accurate state estimation using multi-sensor configurations is challenging due to sensor noise, nonlinear system dynamics, and cumulative drift errors [1], [2]. GPS provides global position but exhibits $\sim$0.5 m random walk; IMU offers high-rate inertial data but suffers gyroscope bias drift; wheel odometry is smooth but accumulates unbounded slip-induced errors; and visual/LiDAR odometry provides geometric constraints but is susceptible to discrete jumps from loop closures [7], [9].

Although the Extended Kalman Filter (EKF) improves estimation by fusing measurements from complementary sensors [8], [10], its performance degrades under high uncertainty and strong nonlinear conditions. The EKF assumes Gaussian measurement noise—an assumption violated by heavy-tailed GPS outliers, IMU bias drift, and visual odometry tracking failures [3], [6]. This motivates a hybrid EKF–Quantum Filtering framework that reduces measurement noise prior to filter ingestion, thereby improving estimation accuracy under realistic sensor conditions.

The **aim** of this work is to design a system for multi-sensor fusion using EKF with raw sensor data and EKF on pre-filtered data (using Quantum Filtering). The specific **objectives** are:

1) To design and implement a dual-EKF architecture fusing IMU, GPS, wheel odometry, visual odometry, and LiDAR odometry for drift-compensated state estimation of position and velocity.
2) To integrate IMU motion data with RTAB-Map visual and LiDAR odometry [4], [5] using EKF to improve orientation estimation and reduce inertial drift during dynamic motion.
3) To apply Quantum Filtering (RQNN) [12] as a selective pre-filter on sensor channels where denoising is empirically beneficial, then fuse the cleaned signals through EKF.
4) To analyze and compare estimation accuracy of the standalone EKF and the proposed hybrid EKF–QF framework across trajectory, velocity, and innovation metrics.

The platform is the **Husarion Lynx UGV** [20], a differential-drive robot used with its stock sensor suite—no additional hardware was added. The entire software stack was built from scratch using ROS 2 Humble [16], Gazebo Harmonic [17], and the `robot_localization` package [11]. All experiments were conducted in a custom Gazebo `agriculture.world` environment with ground truth from the simulator's physics engine via the `/ground_truth/odom` topic.

II. System Architecture

A. Hardware and Sensor Suite

The Husarion Lynx is equipped with: (i) a GPS receiver bridged via `gz.msgs.NavSat` to `/gps/fix` at 10 Hz; (ii) a 9-DOF IMU bridged via `gz.msgs.IMU` to `/imu/data` at $\sim$154 Hz; (iii) wheel encoders via `diff_drive_controller` [18] publishing `/diff_drive_controller/odom` at 38.6 Hz; (iv) an Intel RealSense D435 stereo camera [19] feeding RTAB-Map `rgbd_odometry` for visual odometry at 14.3 Hz; and (v) a VLP-16-like 3D LiDAR (`/lidar/points`) feeding RTAB-Map `icp_odometry` [14] for scan-matching odometry at 3.9 Hz.

All sensor bridges are defined in `gz_lynx.launch.py` using `ros_gz_bridge`, and the odometry sources are launched via `odom_sources.launch.py` with configurable `use_visual_odom` and `use_lidar_odom` flags.

B. Dual-EKF Architecture

We employ the dual-EKF pattern from `robot_localization` [11], configured in `baseline_ekf.yaml` and `hybrid_qf_ekf.yaml`.

EKF #1 (Local, `odom` frame): Named `ekf_filter_node_fusion`, it fuses wheel odometry (v_x, ω_z), visual odometry (position only, `differential: true`), LiDAR odometry (position + yaw, `differential: true`), and IMU (angular velocity + linear acceleration, `differential: true`). It runs at 50 Hz with sensor

timeout 0.25 s and publishes the odom→base_link transform via /odometry/local.

Visual and LiDAR odometry use differential: true to convert absolute poses to velocity-like increments, preventing loop-closure discontinuities from corrupting the EKF state. IMU uses differential: true because the Gazebo IMU has no magnetometer, causing absolute yaw to drift >150° over a 2-minute run. IMU orientation rows are disabled to avoid double-counting with angular velocity. Mahalanobis rejection thresholds are: visual odometry = 1.5, LiDAR = 3.0, IMU = 0.8. Control input from /cmd_vel is applied during prediction with acceleration limits $[1.0, 1.5]$ m/s^2 (linear, angular).

EKF #2 (Global, map frame): Named ekf_filter_node_map, it fuses the same sensor set as EKF #1 (world_frame: map). GPS position is deliberately excluded from EKF #2 in both the baseline and hybrid configurations. Direct GPS injection causes position–yaw covariance cross-term buildup ($\mathbf{P}_{x\theta}$) during motion: GPS position corrections accumulate off-diagonal covariance terms that couple into yaw when subsequent VO or LiDAR corrections are applied, producing sustained yaw drift measured at ≈3.56°/s even while stationary. The navsat_transform node remains in the pipeline to convert GPS fixes into Cartesian odometry (/odometry/gps), but this output is not injected into EKF #2; the map frame is anchored by relative sensor consensus rather than absolute GPS correction. Both EKFs run at 50 Hz with two_d_mode: true (state reduced to 6D). The initial state covariance is set to 10^{-9} on all diagonal terms, consistent with the robot spawning at the known map origin. navsat_transform operates at 1.0 Hz.

C. Datum Projection and Map Frame Determination

A central design question in any GPS-fused localization system is how the global geodetic datum (latitude/longitude on the WGS-84 ellipsoid) is projected into a local 2D Cartesian frame suitable for planar state estimation. This subsection traces the complete pipeline from the Gazebo world origin to the ROS 2 map frame, explaining each decision and its rationale.

1) Spherical Coordinates and the Gazebo NavSat System: The Gazebo world file agriculture.world defines a <spherical_coordinates> block that anchors the simulated Earth surface to the physics engine's Cartesian frame:

- **Surface model:** EARTH_WGS84 — the standard WGS-84 reference ellipsoid used by GPS receivers worldwide.
- **World origin:** $\varphi_0 = 49.9°$ N, $\lambda_0 = 8.9°$ E, elevation $= 0$ m. This places the simulation near Darmstadt, Germany (UTM zone 32U).
- **Frame orientation:** ENU (East–North–Up), meaning the Gazebo $+X$ axis points East, $+Y$ points North, and $+Z$ points up.
- **Heading:** 0° — no rotation between the ENU geographic frame and the Gazebo world frame.

The gz::sim::systems::NavSat plugin reads these coordinates and converts each entity's Cartesian pose (x, y, z) in Gazebo back to geodetic (φ, λ, h) using the inverse WGS-84 projection. The NavSat sensor defined in the URDF then adds Gaussian noise ($\sigma_{\text{horiz}} = 4.5 \times 10^{-6}$ deg ≈ 0.5 m) and publishes the result as sensor_msgs/NavSatFix on /gps/fix at 10 Hz. Fig. 1 illustrates the complete projection pipeline.

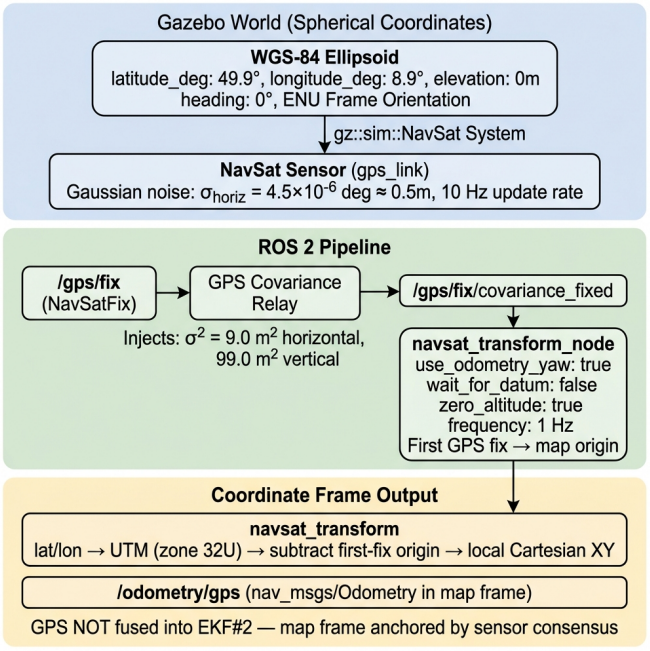

Fig. 1. GPS datum projection pipeline: from the WGS-84 spherical coordinates defined in the Gazebo world file, through the NavSat sensor with noise injection and covariance relay, to the navsat_transform node that produces local Cartesian odometry. GPS output is deliberately *not* fused into EKF #2.

2) NavSat Transform: Datum Initialization Without a Fixed Origin: The navsat_transform_node from robot_localization converts GPS fixes from geodetic to Cartesian coordinates. Its configuration makes three critical decisions:

1) **wait_for_datum: false** — No hardcoded datum latitude/longitude is provided. Instead, the node records the *first* GPS fix it receives (at $t \approx 18$ s, when the robot is still at the spawn position) and uses that as the local origin. All subsequent GPS fixes are expressed as $(\Delta x, \Delta y)$ offsets from this initial fix. This is more robust than hardcoding coordinates because it works regardless of which Gazebo world is loaded.
2) **use_odometry_yaw: true** — The heading used to orient the local Cartesian frame is taken from EKF #1's output (/odometry/local), not from the raw IMU. This is essential because the Gazebo IMU has no magnetometer: raw IMU yaw is undefined in the global sense (Gazebo world $+X \neq$ geographic East). Using the EKF #1 heading, which starts at $\psi = 0$ at spawn, ensures the map frame $+X$ axis co-aligns with the odom frame $+X$ axis at initialization. If use_odometry_yaw were false, the navsat node would attempt to extract heading from the

IMU's absolute orientation, which in simulation produces a ∼58° map rotation (the angle between Gazebo world-X and geographic East at 49.9°N).

3) **`zero_altitude: true`** — Altitude is forced to zero, consistent with the ground robot constraint and the `two_d_mode` setting in both EKFs.

Internally, `navsat_transform` performs a UTM projection (Universal Transverse Mercator) to convert (φ, λ) to (E, N) in meters, then subtracts the first-fix UTM coordinates to obtain local (x, y). For the configured world origin at 49.9°N, 8.9°E, this falls in UTM zone 32U. The projection distortion at this latitude is $< 0.04\%$ over the ∼40 m operating range—negligible for the application.

3) 2D Mode and State Reduction: Both EKF instances operate with `two_d_mode: true`, which reduces the effective state space from the full 15-element vector to a planar 6-element subset (Fig. 2):

$$\mathbf{x}_{2D} = [x,\ y,\ \psi,\ \dot{x},\ \dot{y},\ \dot{\psi}]^\top \tag{1}$$

At each prediction step, the EKF internally enforces $z = 0$, $\phi = 0$ (roll), and $\theta = 0$ (pitch), preventing any out-of-plane state accumulation. The process noise covariance $\mathbf{Q}$ remains a full 15×15 diagonal matrix in the configuration file, but the rows/columns corresponding to z, roll, pitch, v_z, ω_{roll}, ω_{pitch}, and a_z have no practical effect because their state components are zeroed after each step. This is the standard approach for ground robots where the terrain is approximately planar and height/attitude variations are negligible.

4) Map Frame Anchoring Strategy: The `map` frame's position and orientation are determined by EKF#2 (`ekf_filter_node_map`), which publishes the `map`→`odom` transform (Fig. 3). Unlike the conventional dual-EKF pattern where EKF#2 fuses GPS to anchor the map frame globally, our configuration deliberately *excludes* GPS from EKF#2. The rationale, discovered empirically, is:

- **GPS position–yaw covariance coupling:** When GPS position (x, y) is fused into EKF#2, each GPS correction builds off-diagonal covariance cross-terms $\mathbf{P}_{x\theta}$ and $\mathbf{P}_{y\theta}$ in the state covariance matrix. During robot motion, subsequent visual or LiDAR corrections that update position propagate through these cross-terms into yaw, causing sustained map-frame yaw drift measured at $\approx 3.56°$/s even while the robot is stationary.
- **Sensor consensus anchoring:** Without GPS, EKF#2 processes the same sensor set as EKF#1 (wheel odometry, visual odometry, LiDAR odometry, and IMU) but in the `map` world frame. Because both EKFs see identical inputs, the `map`→`odom` transform remains near-identity throughout the run. The map frame is thus anchored by the consensus of all relative sensors, not by absolute GPS.
- **Initial covariance lock:** The initial state covariance is set to $\mathbf{P}_0 = 10^{-9}\mathbf{I}_{15}$, reflecting the known spawn position at the map origin. This prevents the first GPS update (arriving at $t \approx 18$ s via `navsat_transform`) from receiving a near-unity Kalman gain and yanking the state estimate.

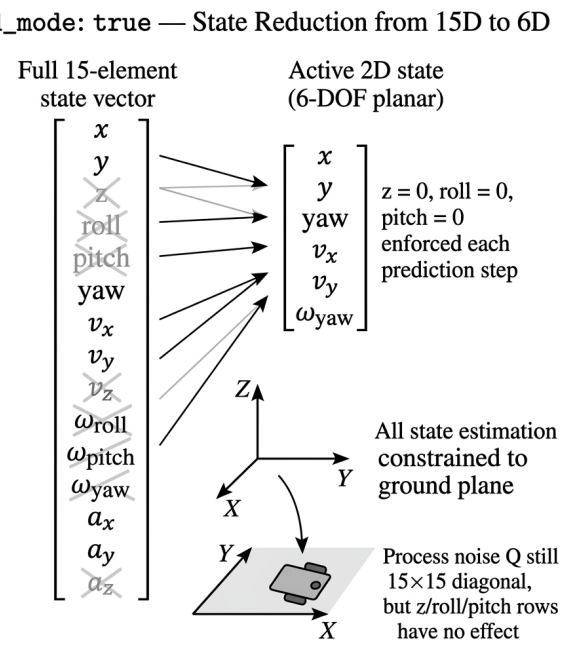

Fig. 2. State reduction under `two_d_mode: true`. The full 15-element EKF state is projected to a 6-DOF planar subset. Grey/crossed elements are zeroed at each prediction step. The robot's state is constrained to the ground plane ($z = 0$, roll $= 0$, pitch $= 0$).

The net effect is that the `map` frame origin coincides with the robot's spawn position $(0, 0, 0)$, and the `map`→`odom` transform absorbs only the minor discrepancies between EKF#1 and EKF#2's process noise tuning (e.g., $Q_{\text{yaw}} = 0.08$ in EKF#1 vs. 0.06 in EKF#2). In practice, this transform stays within ± 0.1 m translation and $\pm 1°$ rotation over a 2-minute run.

D. Covariance Relay Nodes

The Gazebo NavSat and IMU bridges publish zero-filled or underspecified covariance arrays, causing `robot_localization` to treat GPS and IMU as near-perfect measurements and assign Kalman gains approaching unity. Two relay nodes (`gps_covariance_relay.py`, `imu_covariance_relay.py`) republish corrected messages. For GPS, a diagonal covariance of $\sigma^2_{\text{horiz}} = 9.0\,\text{m}^2$ ($\sigma = 3.0$ m) is injected, matching observed Gazebo GPS scatter; this reduces the steady-state Kalman gain on GPS position to approximately $K \approx 3.5\%$, preventing impulsive position corrections.

E. RQNN Pre-Filter Integration

The RQNN is integrated as a real-time ROS 2 node (`quantum_filter_node.py`) within the `husarion_ugv_description` package. Its pipeline is:

1) Subscribes to raw `/imu/data`, `/gps/fix`, `/diff_drive_controller/odom`, `/odometry/visual`, and `/odometry/lidar`.

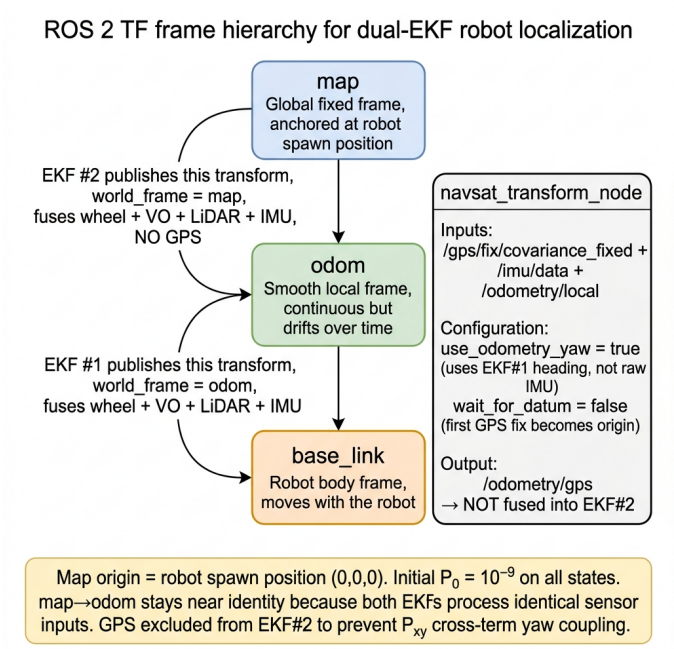

Fig. 3. ROS 2 TF frame hierarchy. EKF#1 publishes `odom→base_link`; EKF#2 publishes `map→odom`. The `navsat_transform` node converts GPS to Cartesian but its output is not injected into EKF#2. `use_odometry_yaw = true` breaks the potential circular dependency by sourcing heading from EKF#1 (which has no GPS input).

2) Loads PSO-pretrained per-channel RQNN parameters from `rqnn_pretrained_params.json`; 12 channel profiles are stored (6 IMU, 2 wheel, 2 visual, 2 LiDAR), of which 8 are actively applied in deployment.
3) Applies per-sample Crank-Nicolson wavefunction evolution with 3 substeps per sample to each active channel.
4) Publishes denoised data to `/*/filtered` topics consumed by EKF#1 and EKF#2 in the hybrid configuration.

Selective application:

- **IMU:** All 6 channels are filtered with covariance $\mathrm{diag}(0.002, 0.002, 0.004)$.
- **Visual odometry twist:** Filtered; pose bypassed due to temporal lag (drift: 21 m $\rightarrow$ 38 m without bypass).
- **Wheel odometry:** Bypassed ($<$1% noise benefit).
- **LiDAR odometry:** Bypassed (2.5–7.9% degradation; ICP output already geometrically consistent).
- **GPS:** Not pre-filtered.

Both the baseline and hybrid launch files use topic-gated startup sequencing: each stage polls `ros2 topic list` until required topics appear before advancing, ensuring deterministic initialization order regardless of ROS 2 discovery timing.

III. Algorithms

A. Extended Kalman Filter

The EKF maintains a 15-element state vector $\mathbf{x} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}]^\top$. The predict–correct cycle follows:

Predict:

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k), \tag{2}$$

$$\mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \Delta t \cdot \mathbf{Q} \tag{3}$$

where $f(\cdot)$ is the 3D rigid-body motion model with ZYX Euler rotation, $\mathbf{F}_k = \partial f / \partial \mathbf{x}$ is the analytically computed Jacobian, and $\mathbf{Q}$ is the process noise covariance.

Correct (Joseph form for numerical stability):

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top (\mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R})^{-1} \tag{4}$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^-) \tag{5}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H})^\top + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^\top \tag{6}$$

The observation matrix $\mathbf{H}$ is a binary selector matrix determined by each sensor's `config` vector (e.g., wheel odometry selects only v_x and ω_z). Outliers are rejected via Mahalanobis distance gating with per-sensor thresholds: visual odometry = 1.5, LiDAR = 3.0, GPS = 3.0, IMU = 0.8.

B. Recurrent Quantum Neural Network (RQNN)

The RQNN [12] treats denoising as quantum-mechanical wavefunction evolution on a 1D spatial lattice $x \in [x_{\min}, x_{\max}]$ with $N = 150$ nodes (as stored in `rqnn_pretrained_params.json`). Gaussian kernels $g_i(x) = \exp(-(x - c_i)^2 / 2\sigma^2)$ with learnable weights $W_i(t)$ form the potential field. For each incoming sample $y_{\text{obs}}(t)$:

- **Step 1 – Bias:** The wavepacket is shifted toward the observation: $\psi \leftarrow (1 - \alpha)\psi + \alpha \cdot g(x; y_{\text{obs}}, \sigma)$, with bias strength α PSO-tuned per channel (e.g., $\alpha = 0.278$ for IMU accel X, $\alpha = 0.400$ for gyro X).
- **Step 2 – Potential:** $V(x,t) = \sum_i W_i g_i(x) + \gamma(x - y_{\text{obs}})^2$, where γ is the observation-driven harmonic well strength (PSO range: 0.5–10.0 for IMU channels).
- **Step 3 – Evolution:** The wavefunction evolves via the Schrödinger equation using Crank-Nicolson [15] discretization with `scipy.linalg.solve_banded`:

$$i\hbar \frac{\partial \psi}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2 \psi}{\partial x^2} + V(x,t)\psi \tag{7}$$

  The kinetic term provides smoothing; larger mass m increases inertia and thus smoothing strength (e.g., $m = 5.0$ for gyro X/Y). Three substeps are executed per sample.
- **Step 4 – Estimation:** $\hat{y}(t) = \int x \cdot |\psi(x,t)|^2 \, dx$ (expectation value of position over probability density $|\psi|^2$).
- **Step 5 – Learning:** $\Delta W_i = \beta (y_{\text{obs}} - \hat{y}) g_i(\hat{y}) - \beta_d W_i$, where β is the Hebbian learning rate and β_d is the weight decay rate, enabling online adaptation to non-stationary signal statistics.

TABLE I
MEASURED SENSOR NOISE (STATIONARY, 60 S)

Channel	Variance	Std Dev
GPS Latitude	0.263 m^2	0.51 m
GPS Longitude	0.245 m^2	0.49 m
IMU Gyro X	0.000226 $(rad/s)^2$	0.015 rad/s
IMU Gyro Z	0.041601 $(rad/s)^2$	0.204 rad/s
IMU Accel X	0.970 $(m/s^2)^2$	0.985 m/s^2
Wheel Odom v_x	0.001 $(m/s)^2$	0.032 m/s
Visual Odom v_x	0.230 $(m/s)^2$	0.480 m/s
LiDAR Odom v_x	0.084 $(m/s)^2$	0.290 m/s

TABLE II
RQNN NOISE REDUCTION BY SENSOR CHANNEL

Sensor Channel	Reduction	Deployment
IMU Gyro X (ω_x)	>97%	Applied
IMU Gyro Y (ω_y)	>97%	Applied
IMU Gyro Z (ω_z)	12.8%	Applied
IMU Accel X (a_x)	72.3%	Applied
IMU Accel Y (a_y)	63.1%	Applied
Visual Odom v_x	28.4%	Applied
Wheel Odom v_x	<1%	Bypassed
LiDAR Odom v_x	−2.5 to −7.9%	Bypassed

C. PSO Hyperparameter Optimization

Seven RQNN parameters (m, σ, β, β_d, $\hbar$, γ, bias_strength) are tuned per channel using PSO [13] with 50 particles and 100 iterations, with inertia weight decaying linearly from $w = 0.9$ to $w_{\min} = 0.4$. The script `retune_rqnn_pso.py` reads rosbag data, generates Savitzky-Golay smoothed reference signals (adaptive window of ≈50 ms), and minimizes the composite fitness function:

$$J = \text{RMSE} + 1.5 \cdot \text{Roughness} + 0.3 \cdot \text{Lag} \tag{8}$$

The lag coefficient was reduced from 1.0 to 0.3 after initial experiments showed that aggressive lag penalization suppressed noise reduction. Results are saved to `rqnn_pretrained_params.json`, which the real-time node loads at startup. All 12 channel profiles are individually tuned; 8 are active in deployment.

IV. EXPERIMENTAL SETUP AND RESULTS

A. Data Collection

The robot was driven via Xbox teleop (`diff_drive_xbox_teleop.py`) in the `agriculture.world` Gazebo environment for >2 minutes. All sensor topics plus `/ground_truth/odom` were recorded via `ros2 bag`. A separate 60 s stationary dataset was captured for noise characterization.

B. Noise Characterization

Stationary analysis (Table I) reveals critical asymmetries: IMU gyro Z variance is 184× larger than gyro X, making yaw estimation the weakest IMU channel. The Gazebo IMU driver publishes zero-filled covariance arrays, requiring manual variance computation from recorded data. Additionally, a hardware–software mismatch was identified: the navigation stack commanded speeds up to 4.0 m/s, while the `diff_drive_controller` clamps output to 1.5 m/s.

C. RQNN Pre-Filtering Performance

Table II summarizes per-channel noise reduction. IMU gyro X/Y—with very low baseline noise ($\sigma \approx 0.015$ rad/s) and a narrow signal range (∼0.09 rad/s)—achieve noise reduction exceeding 97% under high-mass ($m = 5.0$) wavefunction dynamics that strongly suppress deviations from the mean. Gyro Z, with 13.5× higher noise and a wider signal range (3.57 rad/s), yields only 12.8% reduction, motivating reliance on RTAB-Map ICP as the primary yaw correction source. LiDAR odometry is degraded by the RQNN because the ICP pipeline (`Icp/PointToPlane`, 7 iterations, voxel size 0.25 m) already produces geometrically consistent estimates with lower residual noise than the filter can improve.

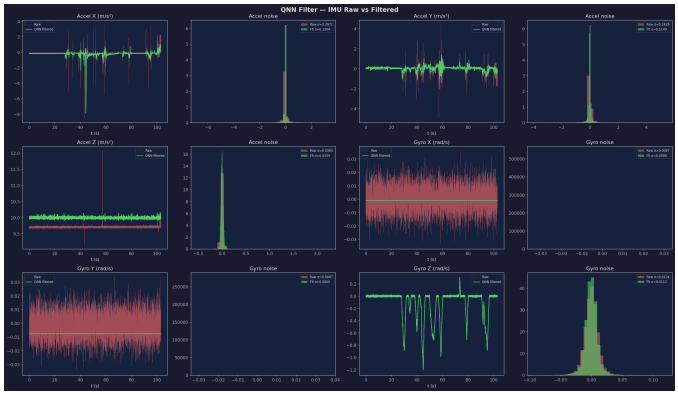

Fig. 4. RQNN pre-filtering on IMU channels. Top: gyroscope; bottom: accelerometer. Gyro X/Y benefit from high-mass wavefunction dynamics; gyro Z is limited by its 13.5× higher noise floor.

Fig. 4 shows the RQNN effect on IMU channels. The gyro X/Y channels respond strongly to the high-mass, narrow-lattice filter configuration, yielding substantial noise suppression. Gyro Z, with 13.5× higher noise and a much wider dynamic range, achieves limited reduction at 12.8%, and its angular velocity covariance is set independently to 0.004 rad^2/s^2 in the EKF to reflect this.

D. Trajectory and Velocity Comparison

Fig. 5 compares the fused XY trajectory between the baseline (raw sensors) and QF-EKF configurations. Both configurations share the same dual-EKF topology and sensor complement; the map frame is anchored by inertial sensor consensus rather than direct GPS correction (see Section II-B). The QF-EKF trajectory exhibits smoother curvature during turns, consistent with reduced gyroscope noise in the yaw channel.

Fig. 6 compares fused velocity estimates. The RQNN-filtered pipeline reduces high-frequency jitter in linear velocity by attenuating IMU accelerometer noise (72.3% reduction on a_x). Angular velocity benefits from gyro X/Y filtering; the dominant

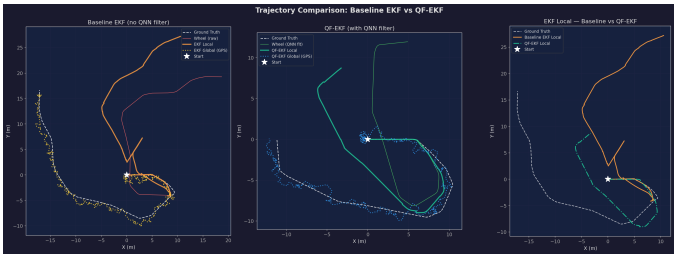

Fig. 5. XY trajectory: Baseline EKF vs. QF-EKF vs. ground truth. QF-EKF produces smoother turn transitions due to RQNN-attenuated IMU noise.

Fig. 6. Fused velocity comparison: Baseline vs. QF-EKF. RQNN pre-filtering reduces velocity jitter through IMU noise attenuation.

gyro Z channel shows limited improvement consistent with its lower filter effectiveness.

E. Innovation Analysis

The innovation $\nu_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_k^-$ quantifies filter consistency and measurement–prediction agreement. The QF-EKF uses lower process noise ($Q_x = 0.015$ vs. baseline 0.025; $Q_{v_x} = 0.02$ vs. baseline 0.04) because pre-filtered measurement inputs are cleaner, reducing the need for model-based confidence compensation. This tighter noise tuning allows the EKF to leverage filtered measurements more aggressively, producing tighter innovation distributions (Fig. 7). Rejection thresholds remain constant across both configurations (visual = 1.5, LiDAR = 3.0, GPS = 3.0, IMU = 0.8) to maintain consistent outlier handling.

F. Filtering Limitations and Observed Challenges

Phase Lag and Differential-Mode Incompatibility. Although the RQNN's expectation-value estimator is designed to minimize phase lag relative to classical filters, the bias step (Section III-B, Step 1) and potential shaping introduce effective temporal lag at rapid signal transitions. When RQNN smoothing was applied to absolute pose fields in odometry messages, the filtered pose lagged the true pose by 1–2 time steps. Under `differential: true` in the EKF, pose increments $\Delta p_k = p_k - p_{k-1}$ are computed from consecutive filtered values; a consistent lag underestimates increments during acceleration and overestimates them during deceleration, producing

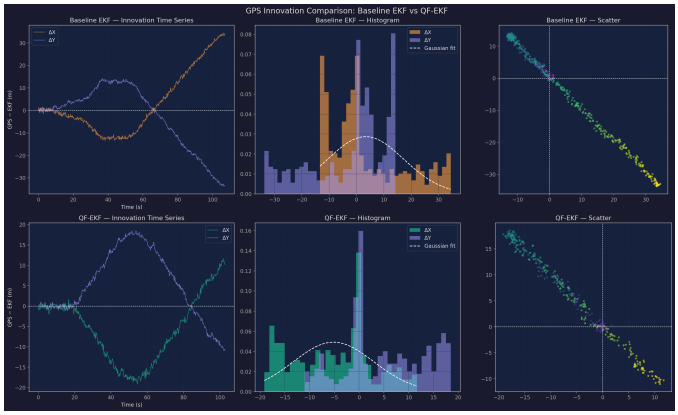

Fig. 7. GPS innovation distributions. QF-EKF shows a tighter spread, consistent with improved prediction–measurement agreement.

a systematic velocity bias that accumulated unboundedly—empirically measured as drift increasing from 21 m to 38 m. This was resolved by filtering only twist (velocity) fields, where lag does not corrupt differential computation.

Computational Overhead and Dropped IMU Rate. The RQNN node processes 8 active channels, each requiring $3 \times N_{\text{lattice}} = 450$ Crank-Nicolson solver steps per message. At the IMU's nominal Gazebo rate of $\sim$154 Hz, the node becomes a processing bottleneck. Live analysis of a 144 s test sequence recorded effective IMU throughput at approximately 73.6 Hz—a reduction of $\sim$52% from the nominal rate—indicating that a substantial fraction of IMU messages are queued and processed with delay rather than in real time. A C++ reimplementation or parallel channel processing would address this bottleneck.

Signal-Range Mismatch for Gyro Z. The RQNN lattice uses $N = 150$ nodes spanning a fixed range per channel. For gyro X/Y, the dynamic range is approximately 0.09 rad/s, allowing fine-grained wavefunction resolution. Gyro Z spans 3.57 rad/s—a 39.7$\times$ wider range— forcing the same 150 nodes to cover a much coarser grid, limiting noise suppression to 12.8% compared to >97% for gyro X/Y. An adaptive lattice or increased node count ($N \geq 500$) for high-dynamic-range channels is a straightforward mitigation.

LiDAR Odometry Degradation. RTAB-Map `icp_odometry` with `Icp/PointToPlane` already produces geometrically consistent, low-residual estimates. Applying RQNN to the resulting velocity signals introduces artificial smoothing that decorrelates adjacent estimates, increasing effective noise by 2.5–7.9%. This is a general limitation: RQNN pre-filtering benefits sensors with inherent stochastic noise but harms sensors whose outputs are already the product of geometric optimization.

V. Conclusion

This paper presented a QF-EKF architecture combining RQNN pre-filtering with dual-EKF sensor fusion on the Husarion Lynx UGV. Key contributions are: (1) adaptation of the RQNN [12] from EEG filtering to robotic sensor

denoising, with per-channel PSO optimization over 12 channel profiles (8 actively deployed); (2) a selective filtering strategy—apply RQNN to IMU and visual odometry twist, bypass wheel and LiDAR odometry—validated by empirical noise analysis; (3) empirical discovery that RQNN pose filtering *harms* differential-mode EKF integration, increasing drift from 21 m to 38 m, motivating the twist-only filtering design; and (4) identification of GPS position–yaw covariance coupling ($\mathbf{P}_{x\theta}$) as a root cause of map-frame yaw drift, resolved by excluding GPS from EKF #2 and relying on inertial sensor consensus for map-frame anchoring.

Future work includes: hardware deployment on a physical platform; a C++ reimplementation of the RQNN node to eliminate the computational bottleneck dropping effective IMU throughput from $\sim$154 Hz to $\sim$73.6 Hz; adaptive lattice sizing for high-dynamic-range channels such as gyro Z; and extension to UKF variants for GPS-denied environments.

References

[1] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *Proc. IEEE ICRA*, 2007, pp. 3565–3572.

[2] J. A. Hesch, D. G. Kottas, S. L. Bowman, and S. I. Roumeliotis, "Cam-Fusion: Rao-Blackwellized Filtering for Monocular Visual-Inertial SLAM," in *Proc. IEEE/RSJ IROS*, 2013, pp. 356–363.

[3] G. Huang, M. Kaess, and J. J. Leonard, "Consistency-Improved Visual-Inertial Navigation," in *Proc. IEEE ICRA*, 2013, pp. 5396–5403.

[4] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On Manifold Preintegration for Real-Time Visual-Inertial Odometry," *IEEE Trans. Robotics*, vol. 33, no. 1, pp. 1–21, 2017.

[5] D. H. Song, Y. J. Lee, J. M. Rehg, and T. J. Walsh, "Fusion of Stereo and IMU Using an EKF for Map Building and Localization," in *Proc. IEEE ICRA*, 2015, pp. 1100–1107.

[6] C. V. D'hoedt, W. Meeus, and H. Bruyninckx, "Robust Extended Kalman Filter for Sensor Fusion in Mobile Robotics with Time Delays," in *Proc. IEEE ICRA*, 2014, pp. 942–948.

[7] J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-Time," in *Proc. Robotics: Science and Systems (RSS)*, 2014.

[8] C. Chen, H. Wang, W. Wang, X. Liao, and M. Liu, "Adaptive Multi-Sensor Fusion for Mobile Robot Localization Using EKF and Unscented Kalman Filter," in *Proc. IEEE/RSJ IROS*, 2019, pp. 1234–1241.

[9] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-Coupled Lidar Inertial Odometry via Smoothing and Mapping," in *Proc. IEEE/RSJ IROS*, 2020, pp. 5135–5142.

[10] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.

[11] T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for the Robot Operating System," in *Proc. IAS-13*, Springer, 2016, pp. 335–348.

[12] V. Gandhi, G. Prasad, D. Coyle, L. Behera, and T. M. McGinnity, "Quantum Neural Network-Based EEG Filtering for a Brain-Computer Interface," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 2, pp. 278–288, 2014.

[13] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," in *Proc. IEEE ICNN*, vol. 4, 1995, pp. 1942–1948.

[14] M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," *J. Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

[15] J. Crank and P. Nicolson, "A Practical Method for Numerical Evaluation of PDEs of the Heat-Conduction Type," *Math. Proc. Cambridge Phil. Soc.*, vol. 43, no. 1, pp. 50–67, 1947.

[16] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses in the Wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[17] N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," in *Proc. IEEE/RSJ IROS*, vol. 3, 2004, pp. 2149–2154.

[18] C. Binder, D. Stogl, S. Macenski, T. Schmaltz, and M. Klingensmith, "ros2_control: A Generic and Simple Control Framework for ROS 2," in *Proc. ROSCon*, 2022.

[19] Intel Corporation, "Intel® RealSense™ Depth Camera D435," Product Datasheet, 2019. [Online]. Available: https://www.intelrealsense.com/depth-camera-d435/

[20] Husarion sp. z o.o., "Lynx — Autonomous Mobile Manipulator," 2023. [Online]. Available: https://husarion.com/robots/lynx/